

AI Wrote the Code — But Can Your Team Own It?

Vibe Coding · Context Engineering · Spec-Driven · Event-Driven Coding Agents

Ron Dagdag

Microsoft AI MVP / Research Engineering Manager at Thomson Reuters

The best AI code isn't the code the AI wrote.

It's the code the AI wrote that your team can own.



Microsoft®
Most Valuable
Professional

Ron Dagdag

Code > Create > Coach > Repeat.

Research Engineering Manager at

“Opinions expressed here are my own.
I am not an agent of my employer”



**Thomson
Reuters™**



Award Categories

AI, Windows Development,
Internet of Things, Mixed Reality

*Have you ever shown an AI demo
that wowed the room...*

...then tried to ship it?



The Demo-to-Production Gap



Inconsistent Code

Different patterns on every generation



Missing Tests

Works today, breaks tomorrow



Security Gaps

Blind spots the AI doesn't flag



Unownable PRs

Nobody wants to review or maintain it

Where AI Actually Helps (and Where It Doesn't)



+35–40%

Low Complexity + Greenfield

CRUD, boilerplate, scaffolding

+15–20%

Low Complexity + Brownfield

Familiar patterns, existing codebase

+10–15%

High Complexity + Greenfield

Novel architecture, hard problems

+0–10%

High Complexity + Brownfield

Legacy code, deep domain knowledge

← Greenfield

Brownfield →

Four Approaches to AI-Assisted Development



Vibe Coding

Fast, intuitive



Context Engineering

Consistent, informed



Spec-Driven Dev

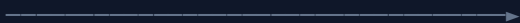
Governed, reviewable



Event-Driven Coding Agents

Async, like a teammate

Exploration



Production at Scale

The GPS Analogy



Vibe Coding

"Take me somewhere nice"

You might end up at a great
restaurant.
Or a parking lot.

Context Engineering

*"Downtown, avoid
highways,
I like Italian"*

You'll get good options.
Mostly right.

Spec-Driven

*"123 Main St, arrive by
5pm,
via I-95, no tolls"*

Exact destination. Verified.
Repeatable.

What the AI Actually Sees



✓ Inside the Context Window

Your prompt

Skills / agents.md / CLAUDE.md

Schema files you included

Code examples you provided

The file currently open

✗ Invisible to the AI

3 years of git history

Architecture docs on the wiki

Last week's design decisions

Your team's implicit conventions

That bug fix from Tuesday

The AI can only reason about what you put in the window. Context engineering is choosing what goes in.

The Iceberg of AI Output Quality

10% · What most developers optimize

Your Prompt

▼ 90% · What actually determines output quality

Context Files (CLAUDE.md · agents.md · .cursorrules)

Schemas & Types (OpenAPI · TypeScript interfaces · DB schemas)

Golden Examples (2-3 reference files: 'write code that looks like this')

Permissions & Constraints (what the AI can and can't touch)

Formal Spec (acceptance criteria · test scenarios · non-goals)

Prompt engineering optimizes the 10%. Context engineering owns the 90%.

Optimize Your Context Window

Four dimensions that determine AI output quality



Correctness

Is the context accurate?

Outdated docs = hallucinated
code.

Wrong schemas = wrong output.



Completeness

Does the AI have everything it
needs?

Missing auth rules = missing
auth.

Missing examples = inconsistent
style.



Size

Too much context = noise.

Too little = gaps.

Curate, don't dump.



Trajectory

Does the context guide the AI
toward your goal? Include not
just

WHAT but WHERE you're going.

Our Demo App: VoteJam

A party playlist voting app — same app, three ways to build it



Submit songs

Name + artist

Vote up/down

One vote per user per song

Live rankings

Songs sorted by votes

Auth required

Only logged-in users vote

React + Node/Express + PostgreSQL

Our Journey Today

01

Vibe Coding

What it is + live demo with Lovable

02

Context Engineering

Skills, Knowledge Items & Artifacts in Google Antigravity

03

Spec-Driven Dev

Contracts & CI pipelines with spec-kit + Claude Code

04

Event-Driven Coding Agents

Copilot coding agent + @claude in GitHub

05

The Playbook

When to use each + templates for your team

01

Vibe Coding

Fast, intuitive, conversation-driven AI coding

What Is Vibe Coding?

*"You describe what you want in natural language.
The AI writes the code. You iterate by vibing."*

- **Conversational prompts — describe, don't specify**
- **Rapid iteration by feel — adjust on the fly**
- **Zero upfront planning — start from a blank slate**
- **AI has broad freedom to interpret your intent**



Vibe Coding Tools

Lovable

DEMO

Full-stack app generation from prompts. Deploys instantly.

v0 by Vercel

UI component generation. React + Tailwind output.

Bolt.new

Browser-based AI app builder with full environment.

Replit Agent

End-to-end app creation in Replit's cloud IDE.

Claude Artifacts

Interactive prototypes inside Claude conversations.

ChatGPT Canvas

Code editing and iteration in OpenAI's interface.



Live Demo: Vibe Coding with Lovable

Building VoteJam from a chat prompt

Speed | No tests | No conventions | No owner

Vibe Coding: The Verdict



Use it for

- Spikes & prototypes
- Learning new APIs/tools
- Throwaway scripts
- Creative exploration



Don't use it for

- Production features
- Security-sensitive code
- Long-lived codebases
- Anything that needs code review

02

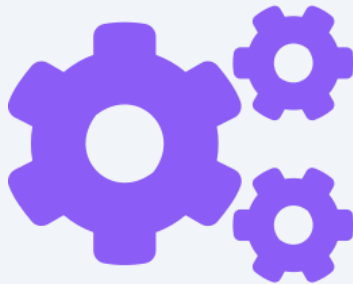
Context Engineering

Giving the AI the right information at the right time

What Is Context Engineering?

"Designing the information environment around the AI so it produces the right output by default."

- **Feed it your documentation & style guides**
- **Include API schemas and type definitions**
- **Set tool permissions and constraints**
- **Provide examples of correct output**





The New Hire Analogy

Day 1 — no onboarding

"Here's the ticket. Go build it."

Invents their own naming conventions

Guesses which patterns to follow

Misses security rules nobody told them

Uses a different framework than the team

Code review: painful and slow

Day 1 — full onboarding packet

Style guide, schemas & code examples

Follows established naming conventions

Applies team patterns automatically

Security rules explicitly stated

Auth middleware added without asking

Code review: spec check, not line-by-line

Context engineering = the onboarding packet you give to the AI.

The Four Pillars of Context



Docs & Standards

Style guides,
architecture docs,
naming conventions



Schemas & Types

API specs, TypeScript
types, DB schemas



Permissions

What the AI can and
cannot touch



Examples

Gold-standard code the
AI should emulate

The Three-Phase Agent Workflow

Good research → Good plans → Good code (bad research → bad everything)

1

Research

Read codebase, schemas,
docs, existing patterns.
Output: research.md

CLAUDE.md • MCP Tools
Builtin Tools • Grep/Read

HUMAN: Review research



2

Planning

Design approach, identify
files to change, plan tests.
Output: plan.md

research.md • Task()
Subagent calls

HUMAN: Review plan



3

Implementation

Write code, run tests,
iterate until green.
Output: working code

plan.md • Write/Edit
Bash • Test runner

HUMAN: Review PR

Context Engineering Tools

Google Antigravity

DEMO

Skills, Knowledge Items, Artifacts. Agent-first IDE.

GitHub Copilot

Agents.md, subagents, MCP servers in VS Code.

Claude Code

CLAUDE.md for project context. Terminal-based AI.

Cursor

.cursorsrules for project-level AI instructions.

Windsurf

Cascade agent with codebase-wide context awareness.

Aider

CLI tool. Git-aware. Repo map for context.



Live Demo: Context Engineering

Google Antigravity + Skills + Knowledge Items + Artifacts

Consistency | Persistent memory | Transparent reasoning

Same Feature. Same AI. Different Context.

Vibe Coded (Lovable)

- No tests — zero coverage
- Patterns change each generation
- No spec to review against
- PR ownership unclear
- Works — but nobody can own it

Context Engineered (Antigravity)

- Auth middleware applied automatically
- Server-side rate limiting
- Tests generated alongside code
- Follows team naming conventions
- Reasoning documented in Artifacts

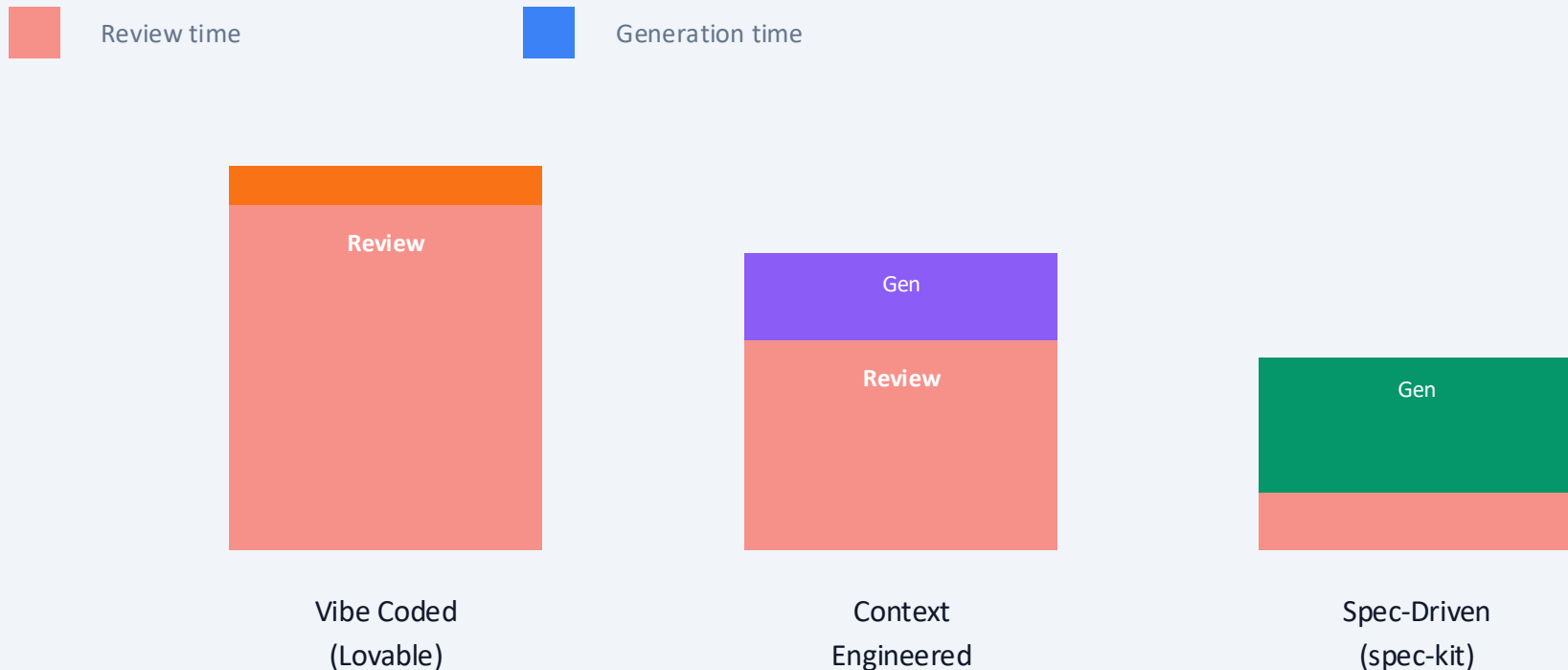
Code Review: Hierarchy of Needs



Source: @blakesmith — "Code Review Essentials for Software Teams"



The Hidden Bottleneck: Code Review



AI made generation fast. Code review is now the bottleneck — and specs are the unlock.

03

Spec-Driven Development

Contracts, checklists, and CI-enforced pipelines

What Is Spec-Driven Development?

*"Write the contract before writing the code.
Let AI generate implementation from verifiable specs."*

- Specs define WHAT, AI generates HOW
- Every PR has a traceable spec origin
- CI checks enforce contract compliance
- Humans review specs, not every line of code



The Division of Labor



Humans Excel At

- Defining requirements
- Setting constraints & trade-offs
- Reviewing spec compliance
- Making architectural decisions



AI Excels At

- Generating boilerplate & plumbing
- Implementing known patterns
- Writing test cases from specs
- Handling repetitive transformations



Anatomy of a Good Spec

1

Interface Contract

Input/output types, endpoint signatures, data shapes — the API the AI must satisfy

2

Acceptance Criteria

Testable, specific conditions. Not 'it should work' — verifiable outcomes per scenario

3

Constraints

Security rules, performance budgets, compliance requirements — the guardrails

4

Non-Goals

Explicitly what you're NOT building. Prevents AI from scope-creeping into a 6-hour impl

5

Test Scenarios

Happy path + edge cases. Write these in the spec — they become the actual test file

Spec-Driven Tools

spec-kit + Claude Code

DEMO

Spec-to-code pipeline via slash commands in VS Code.

Devin

Autonomous AI dev. Takes specs, generates validated PRs.

Factory AI (Droids)

Enterprise AI agents following specs and org standards.

BMAD

Research agent. GitHub issues to validated patches.

OpenHands

Open-source AI dev platform. Spec-driven execution.

Codegen

Codebase-aware agent with full repo context.

The Spec-Driven Pipeline

1

Write Spec

Interfaces, criteria,
constraints

2

Gen Tasks

AI decomposes into
work items

3

Implement

Code from spec
+ context

4

CI Validates

Tests, lint,
security, types

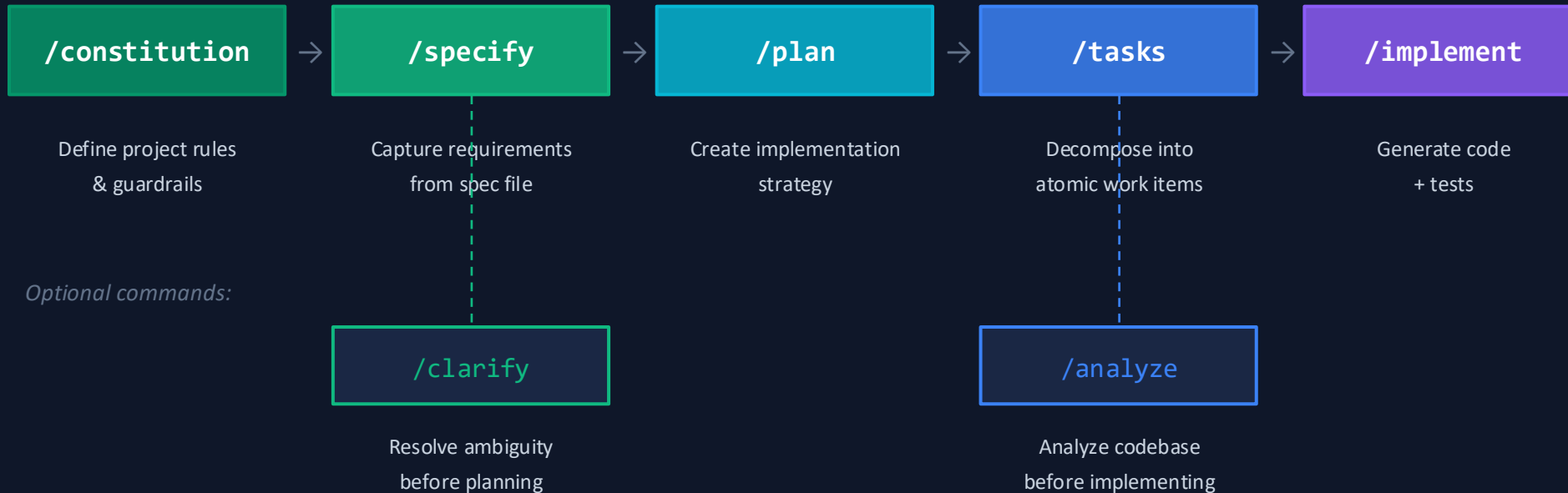
5

Review

Check spec
compliance

The spec-kit Commands

Run these slash commands inside Claude Code — in order.





Live Demo: Spec-Driven with spec-kit

VS Code + Claude Code — 60-line spec to 400+ lines of validated code

Contract | CI enforcement | Reviewable PRs

Spec-Driven: The Verdict



Use it for

- Production features with compliance needs
- Long-lived, team-maintained code
- Anything requiring a real security audit
- Features where review cost matters most



Skip the formal spec for

- Quick experiments & throwaway scripts
- When there's no prior type system or schema
- Solo/no-review projects
- Early-stage exploration work

04

Event-Driven Coding Agents

AI that works on issues and PRs while you sleep



The Night Shift Teammate

Today: You're in the loop

You write a prompt

AI responds

You read & evaluate

You re-prompt or edit

You paste code in editor

Repeat for every task

Blocked when you're offline

Event-Driven: You review

You write a clear GitHub issue

You close your laptop

Agent researches codebase

Agent plans & implements

Agent opens a draft PR

You come back to a review

Works while you sleep

The shift: from CODER who uses AI → MANAGER who reviews AI output.

The Next Frontier: Event-Driven Coding Agents

"Assign an issue. Wake up to a PR."



- AI agents work asynchronously — like a teammate, not a tool
- They research your codebase, plan an approach, implement, and open a PR
- You review and merge — the agent handles the implementation
- Two flavors today: GitHub Copilot coding agent + @claude in GitHub



What 'Event-Driven' Actually Means

The agent sleeps until an event wakes it up — you never open a chat window

EVENT

Issue Assigned

Assign to Copilot
in the issue sidebar

Copilot Coding Agent

Reads codebase →
Plans approach →
Opens Draft PR

EVENT

@claude mention

Type @claude in any
issue or PR comment

@claude GitHub Action

Reads thread →
Investigates code →
Responds or commits

EVENT

PR Comment

'@claude fix this
endpoint validation'

@claude GitHub Action

Reads comment →
Pushes fix commit →
Updates PR branch

'Event-driven' = triggered by work items in your existing workflow, not by opening a chat window.

What Each Approach Adds

Each layer solves a different problem — and includes the previous layers

Vibe Coding

Speed
Creativity
Exploration

No patterns
No tests
No boundaries
No verification

Context Eng

+ Consistent patterns
+ Team conventions
+ Schema awareness
+ Auth by default

No contract
No CI gates
You're still in the loop

Spec-Driven

+ Formal contract
+ CI validation
+ Reviewable PRs
+ Acceptance tests

Still synchronous
You drive each step
One task at a time

Coding Agents

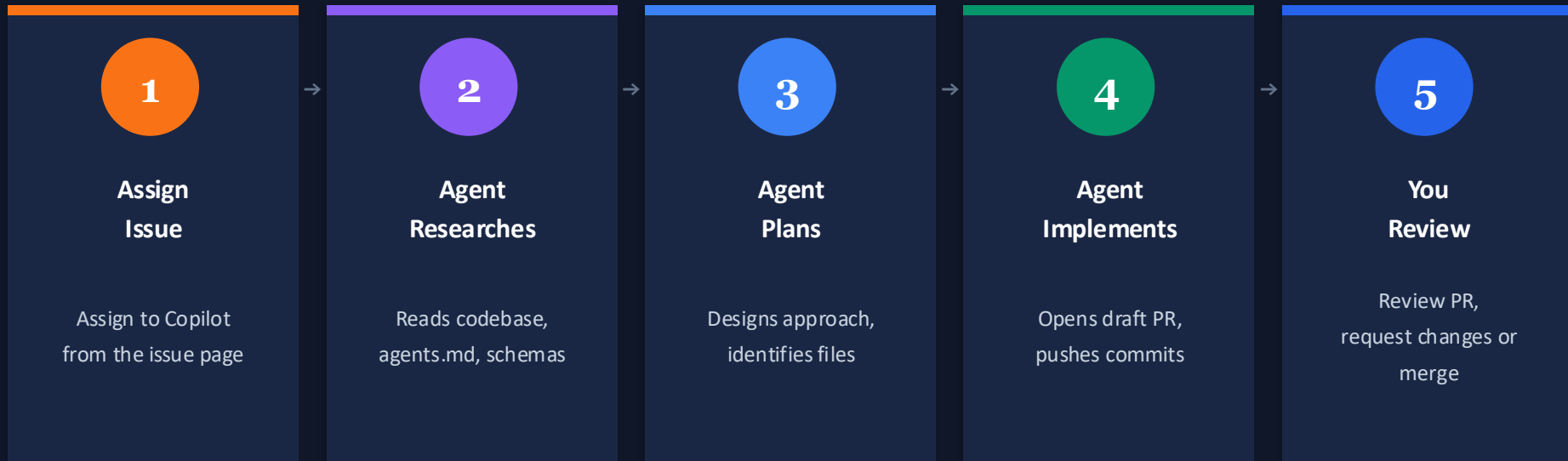
+ Async execution
+ Works on issues/PRs
+ Parallel tasks
+ Agent-created PRs
+ You review, not write

Needs good context
Needs clear issues

Green = what this approach adds. Red = what's still missing.

GitHub Copilot Coding Agent

Built into GitHub — assign an issue, get a PR



Setup: `copilot-setup-steps.yml` workflow + `agents.md` + Copilot Enterprise/Business

The agent runs in a GitHub-hosted environment. Your context files (`agents.md`, `schemas`) travel with the repo.

Write Issues Agents Can Execute

Your issue IS the spec for the agent — quality in, quality out

✗ Vague Issue (agent guesses)

Fix the search.

The search doesn't
work properly.

Please fix it.

✓ Agent-Ready Issue

Add song search to VoteJam

User can search by title or artist.

Criteria:

- Case-insensitive match
- Instant results (no submit)
- Empty query = all songs

Files: routes/songs.ts

Tests required.

Think of the issue as a spec: user story + acceptance criteria + file hints = agent that delivers.



Live Demo: Copilot Coding Agent

Assign an issue → Agent opens a PR

Agents Tab | Async | Issue → PR | Auto-review

@claude in GitHub

Mention @claude in any issue or PR — Claude investigates, answers, or codes

In Issues

@claude implement this feature
based on the issue description

→ Claude creates a PR
with full implementation

*"@claude add a search
endpoint for songs"*

In PR Reviews

@claude fix the TypeError
in the user dashboard

→ Claude pushes a fix
commit to the PR

*"@claude this function
needs error handling"*

For Questions

@claude how should I
implement auth here?

→ Claude analyzes code
and explains approach

*"@claude why is this
test flaky?"*



Live Demo: @claude in GitHub

Mention @claude in an issue → Claude creates a PR

@claude | Issues | PR Reviews | Questions

05

The Playbook

When to use each approach + templates for your team

When to Use What

	Vibe Coding	Context Eng	Spec-Driven	Coding Agents
Speed	Fastest	Fast	Moderate	Async (hours)
Consistency	Low	Medium-High	High	High
Governance	None	Informal	Formal	Automated
Risk Tolerance	High	Medium	Low	Low
Best For	Exploration	Feature dev	Production	Issue backlogs
Review Cost	High	Medium	Low (spec)	Agent-created PRs
Key Artifact	Prompt	agents.md	Spec file	GitHub Issue

Match the approach to the risk profile of the task.

The Maturity Journey

Today

Next Day

Next Week

Next Month

Next Quarter



Ad-hoc prompting
No standards

Add agents.md
to your repo

Context packages
for each service

Spec templates
CI enforcement

Event-driven agents
Issue → PR pipeline

Build the ladder one rung at a time. Each step includes the previous ones.



But... (Addressing the Pushback)

"Writing specs is too slow."

→ A spec takes 20 min. Review of unspecced code takes 4 hours. Do the math.

"The AI should figure this out itself."

→ The AI is a spotlight, not a mind reader. Design the context; don't hope for magic.

"This is over-engineering."

→ Vibe code prototypes, spec production code. Match the formality to the stakes.

"Our team won't adopt this."

→ Start with one agents.md file. Show the output difference. Let the code sell it.

Your Monday Morning Action

1

Audit your current AI usage

List every place your team uses AI coding tools. Classify each as vibe / context / spec.

2

Add a context file to one repo

Create agents.md or CLAUDE.md with your team's patterns, conventions, and 2-3 golden examples.

3

Write one spec for your next feature

Use the 5-part template. Let AI implement it. Compare that PR to your last vibe-coded PR.

Three Takeaways

1

Match the method to the risk

Vibe for exploration, context for delivery, specs for production.

2

Context is the multiplier

Same AI + better context = dramatically better output.

3

Let agents work while you sleep

Assign issues, review PRs. AI agents handle the implementation.

AI Wrote the Code — But Can Your Team Own It?

*The best AI code isn't the code the AI wrote.
It's the code the AI wrote that your team can own.*

<https://github.com/rondagdag/ai-wrote-code-demo>

